Kate Lee

March 11, 2026

IT FDN 110 GP

Assignment 07

# Course Registration Program: OOP Growth Spurt

## Introduction

Module 07 takes the course registration program and gives it a more object-oriented structure. Instead of storing each registration only as a dictionary row, we now convert the data into Student objects built from a parent Person class. That means the program still reads and writes JSON, but now the data in memory is organized through classes, properties, and inheritance.

## Steps I Took

### 1. Reviewed the Starter File & Conceptual Pipeline

In reviewing the starter file, the main changes from Assignment 06 were adding two data classes, Person and Student, using properties for validation, and converting between JSON dictionary rows and Student object rows. The flow chart in my head looks like the following:

```
Enrollments.json
      │
      │ json.load()
      ▼
List of Dictionaries
[
  {"FirstName":"Jeong","LastName":"Lee”,"CourseName":"Carpentry 100"},
  {"FirstName":"Chae","LastName":"Lee","CourseName":"Data Science 200"}
]
      │
      │ FileProcessor.read_data_from_file()
      ▼
Student Objects (in memory)
[
  Student("Jeong","Lee","Carpentry 100"),
  Student("Chae","Lee","Data Science 200")
]
      │
      │ Program runs
      │ (menu, input, display)
      ▼
Updated Student Objects
[
  Student("Jeong","Lee","Carpentry 100"),
  Student("Chae","Lee","Data Science 200"),
  Student("Kate","Lee","Python 100")
]
      │
      │ FileProcessor.write_data_to_file()
      │ convert objects → dictionaries
      ▼
List of Dictionaries
[
  {"FirstName":"Jeong","LastName":"Lee","CourseName":"Carpentry 100"},
  {"FirstName":"Chae","LastName":"Lee","CourseName":"Data Science 200"},
  {"FirstName":"Kate","LastName":"Lee","CourseName":"Python 100"}
]
      │
      │ json.dump()
      ▼
Enrollments.json
```

## 2. Created the Person and Student Classes

I created a Person class with `first_name` and `last_name` properties, then created a Student class that inherits from Person and adds a `course_name` property. This follows the module notes and the work we did in the labs, which taught us that inheritance is a way to reuse common code instead of repeating it. Here is my Student class inheriting from Person:

```python
class Student(Person):
    """A class representing student registration data."""

    def __init__(self, first_name: str = "", last_name: str = "", course_name: str = ""):
        super().__init__(first_name=first_name, last_name=last_name)
        self.course_name = course_name

    @property
    def course_name(self):
        return self.__course_name.title()

    @course_name.setter
    def course_name(self, value: str):
        if isinstance(value, str):
            self.__course_name = value
        else:
            raise ValueError("The course name should be a string.")

    def __str__(self):
        return f"{self.first_name},{self.last_name},{self.course_name}"

```

## 3. Added Properties with Simple Validation

The assignment required properties for `student_first_name`, `student_last_name`, and `course_name`. In code, those ended up as `first_name` and `last_name` in the Person class and `course_name` in the Student class. The name setters use simple validation to reject numbers, while the `course_name` setter checks that the value is a string.

## 4. Implemented the Required CSV-Style Extraction Method

Both classes override `__str__()`, and the Student version returns a comma-separated string of the student's first name, last name, and course name. That made menu option 2 especially convenient, because each row can be displayed by simply printing the Student object.

## 5. Converted JSON Data into Student Objects

When the program starts, `FileProcessor.read_data_from_file()` uses `json.load()` to read a list of dictionary rows from Enrollments.json. It then converts each dictionary into a Student object before appending it to the students list. This matches the assignment requirement that the running program use a two-dimensional list of Student objects, not just raw dictionaries.

### 6. Converted Student Objects Back into JSON

On menu choice 3, `FileProcessor.write_data_to_file()` loops through the Student objects, creates a list of dictionaries, and then saves that list with `json.dump()`. This is necessary because the json module can work with dictionaries and lists, but not directly with custom Student objects.

### 7. Kept Structured Error Handling

I kept structured error handling in the same four places the assignment asked for: reading the file, entering the first name, entering the last name, and writing the data back to the file. The starter file already had `IO.output_error_messages()`, so I used that method consistently whenever an exception block was involved.

## Source Control

GitHub Repository Link:
https://github.com/katherinelee703/IntroToPython/tree/main/Assignment07

## Summary

This assignment helped me understand how classes can make the same program feel more organized and intentional. The finished version still loads registrations from Enrollments.json, lets the user add more (with validation inside the class), displays them, and saves them back to the file. The big difference is that now the program works with Student objects instead of plain dictionary rows, which makes the code feel more like a real system. It's like we're able to build parts of a cohesive program with reusable, Lego-like structures.

## Works Cited

OpenAI. (2026). ChatGPT (GPT-5.2 Thinking). Used for code troubleshooting and written outline assistance.

University of Washington. (2026). Introduction to Programming with Python: Module 07 Notes.